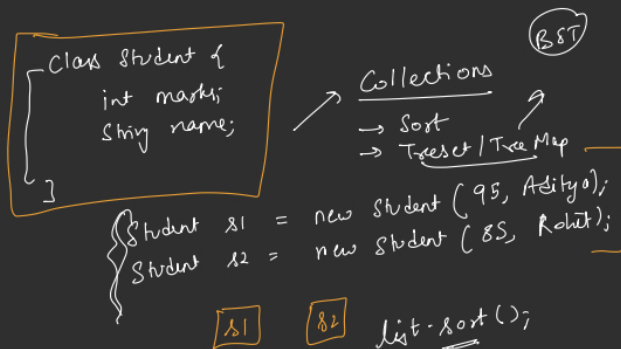
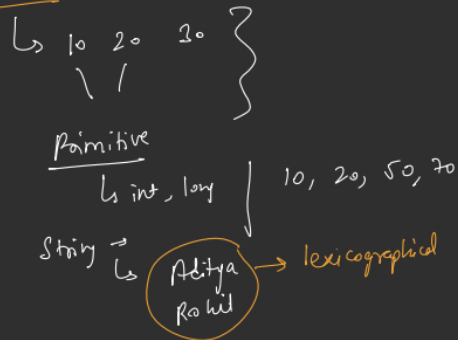


Comparable Interface

Java



Comparable Interface

- ↳ Contract
- ↳ I know how to compare myself with another object of my type.

```
interface Comparable<T> {  
    int compareTo(T o);  
}
```

↳ Functional Interfaces

```
public int compareTo(Student other) {  
    return this.marks - other.marks;  
}
```

a.compareTo(b)

compareTo

s1.compareTo(s2);

s1 → Aditya, 10
s2 → Rohit, 20

return [this.marks - other.marks]

$$= 10 - 20 = -10$$

[Aditya, 10] [Rohit, 20]

s1 → 20

s2 → 10

$$20 - 10 = 10$$

[Rohit, 10] [Aditya, 20]

s1.compareTo(s2);

s1 → Aditya, 10
s2 → Rohit, 20

return [other.marks - this.marks]

$$= 20 - 10 = 10$$

[Rohit, 20] [Aditya, 10]

s1 → 20

s2 → 10

$$10 - 20 = -10$$

[Aditya, 20] [Rohit, 10]

s1.compareTo(s2)

↳ (-ve) [s1 come before s2]

↳ (+ve) [s1 come after s2]

↳ 0 [s1, s2] [s2, s1]

Student

↳ sort

↳ TreeSet / TreeMap

↳ compareTo

String

wrapper class

↳ Comparable

Comparable

↳ Natural Ordering.

Compare By Marks()
Compare By Name()

Compare To()

Sort()

One class → one Natural ordering

int compareTo(T o) {

Boolean → T
Boolean → F

-ve +ve zero

Enum → SMALLER
Enum → LARGER
Enum → EQUAL

81 82

Dangers of 0 return

compareTo() {
81 82
}

+ve 0 -ve

81 82 equal

Tree Set
Tree Map

T F

equals()

compareTo

Duplicates (x)

Rule :

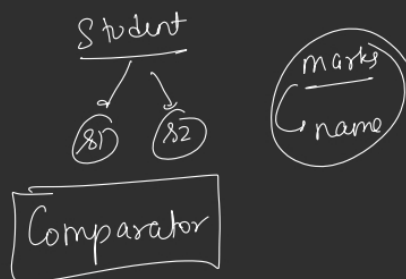
↳ If $a.compareTo(b) == 0$

make sure, $a.equals(b) \rightarrow \text{True}$

Comparable (X)

↳ Custom class

Natural Ordering → obvious



Collections.sort(list);

• max(list);

• min(list);

• fill(list, 0);

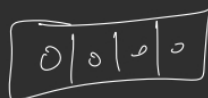
• reverse(list);

• shuffle(list);

• swap(list, a, b);

• binarySearch()

• frequency(list, 5);



$O(19n)$

Collections.unmodifiableList(list)

↓
List read-only

unmodifiableSet()

unmodifiableMap()

Collections.emptyList()

emptySet()

emptyMap()